

Week 08

Introduction to UVM (Universal Verification Methodology)

Jesus Sanchez

Dept. d'Arquitectura de Computadors
Universitat Politècnica de Catalunya

Processor Verification

Master in Innovation and Research in Informatics

Objectives of the Week

- Understand the **purpose and benefits** of using UVM in modern testbenches
- Describe the **UVM base class library** structure and its main components
- Explain **UVM phases** and how simulation progresses
- Build a simple **UVM testbench** using key components
- Write and execute **UVM sequences and tests**

Agenda

1. Motivation and basics of UVM
2. Core UVM classes
3. Simulation phases and communication
4. Minimal UVM example walkthrough
5. Wrap-up and self-learning

Why UVM?

■ Limitations of Ad-Hoc / Homegrown Testbenches

- Poor reusability
 - Custom testbenches tied to a specific DUT $\Rightarrow$ Difficult to reuse across projects or teams
- Lack of standardization
 - Teams use different structures, making collaboration and onboarding harder
- Scalability issues
 - Hard to manage verification complexity for large designs (e.g., CPUs, SoCs)
- No separation of tasks
 - Test logic, stimulus generation, and checking are often tightly coupled
- Manual coordination
 - Synchronization between components (e.g., driver, monitor) is often fragile

From SystemVerilog to UVM

• Pre-UVM Era

- Handwritten Testbenches
 - Fully manual, low reuse
 - Limited structure, no standard practices
- eRM & AVM (Mentor), VMM (Synopsys)
 - Vendor-specific methodologies
 - Introduced concepts like agents, environments, factories
 - Still incompatible across vendors



• The Emergence of SystemVerilog (IEEE 1800)

- Unified design and verification language
- Key features: class, randomize, interface, assertions, covergroups
- Enabled advanced methodologies but needed a standard framework



• Birth of UVM (Universal Verification Methodology)

- Timeline
 - 2011: UVM 1.0 released by Accellera
 - 2013: UVM 1.1
 - 2015: UVM 1.2
 - 2017: UVM becomes IEEE standard 1800.2-2017
- Unified concepts from VMM, OVM, and others
- Fully built on top of SystemVerilog classes
- Widely adopted by EDA vendors and the industry



• Today

- UVM is the de-facto industry standard for verification of complex digital designs
- Supported by Cadence, Synopsys, Mentor/Siemens, etc.
- Still evolving: UVM-1.2, UVM IEEE 1800.2-2020

What is UVM?

■ Definition

- UVM is a **standardized, modular, and reusable** SystemVerilog-based **verification framework**
- Developed and maintained by **Accellera** (now IEEE 1800.2-2020, [here](#))

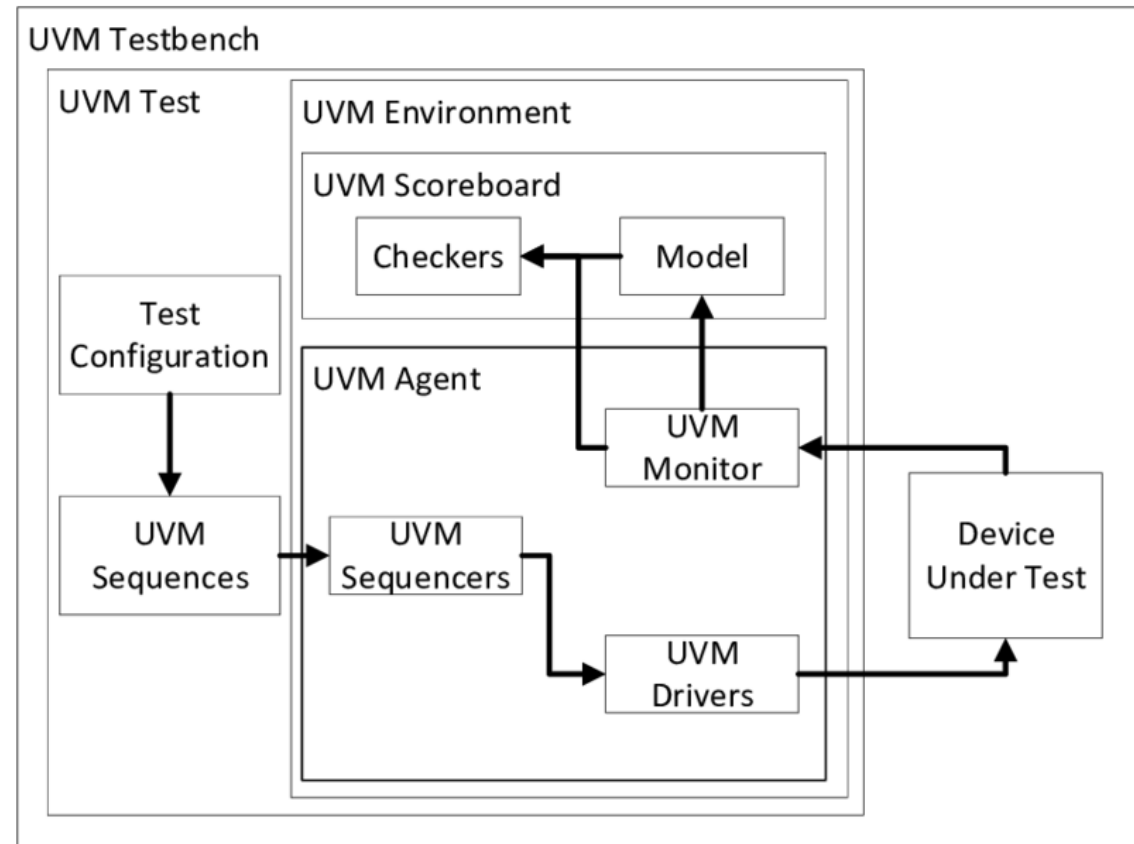
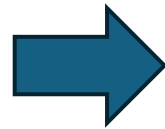
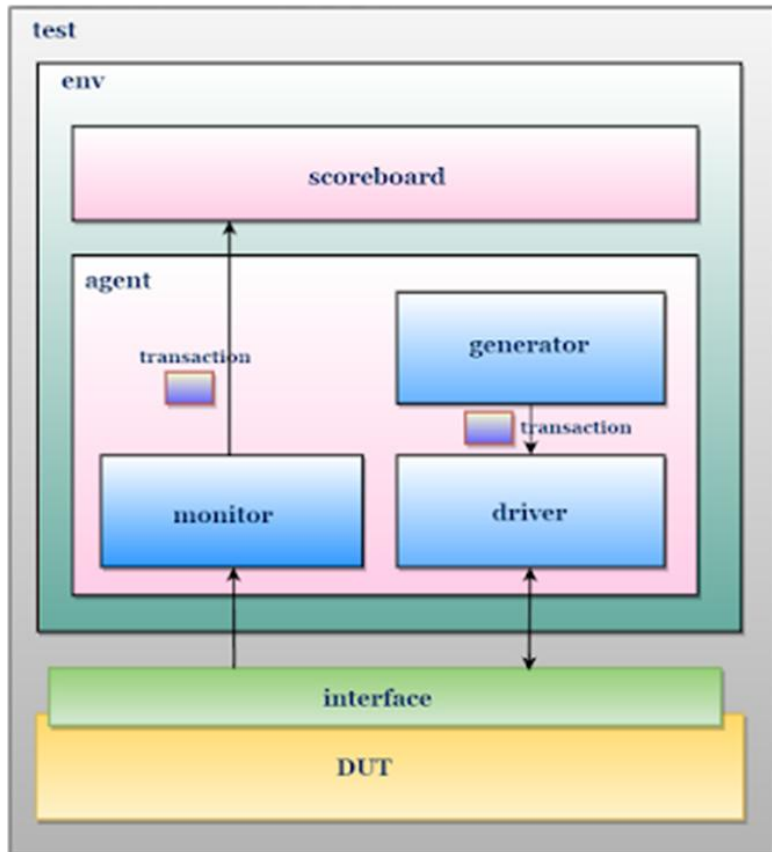
■ Key Characteristics

- Built entirely in **SystemVerilog classes**
- Provides a **rich library** of base classes and utilities
- Encourages **code reuse, scalability, and maintainability**

■ UVM Includes

- Base classes for **testbench architecture**: test, env, agent, driver, monitor, sequencer, etc.
- Transaction-level modeling (TLM)
- **Configuration database** and **factory** for component control
- **Phase-based simulation control + Reporting, objection, and messaging** utilities

Structure of a UVM Testbench – Big Picture

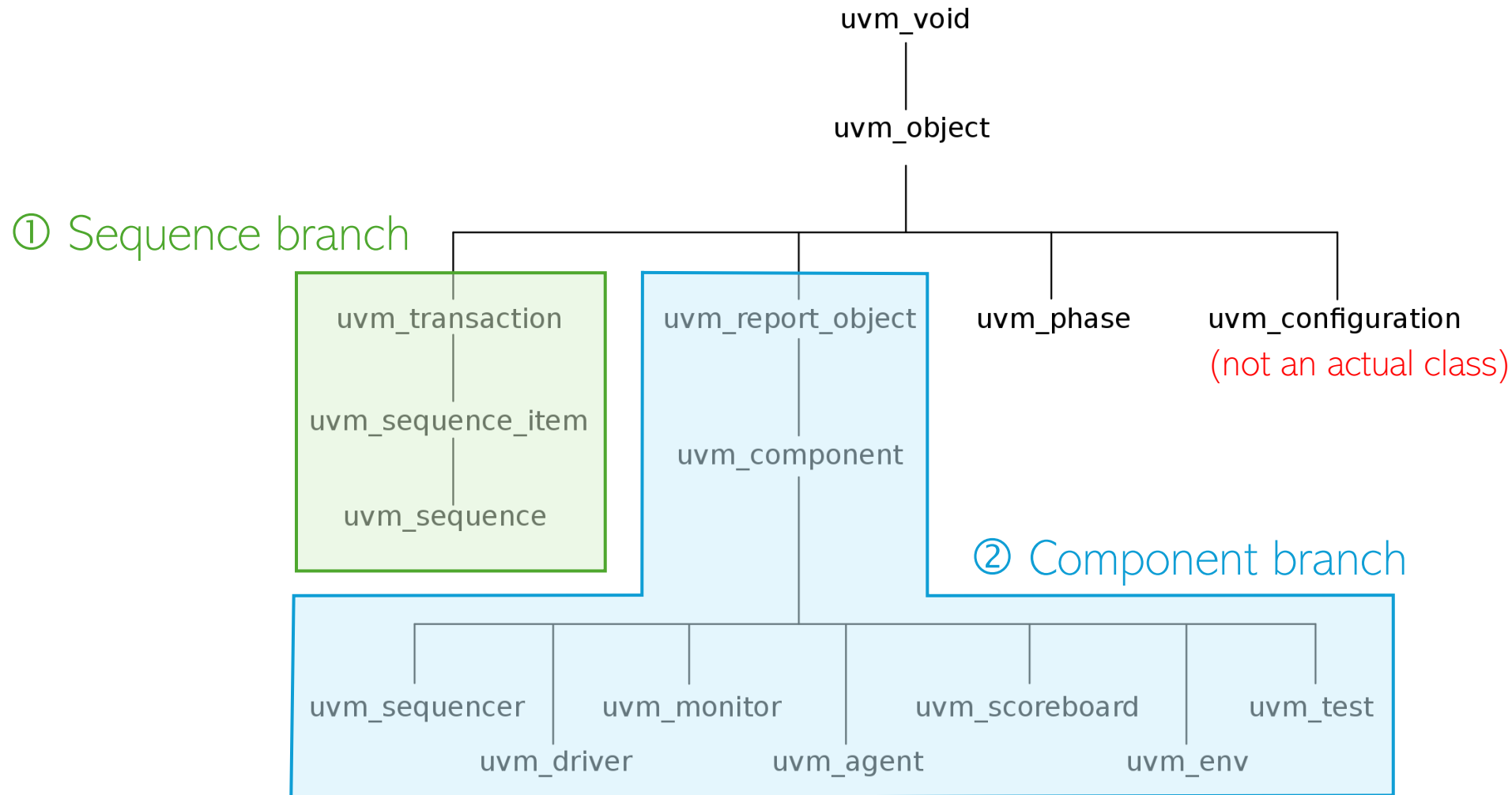


A UVM testbench introduces discipline, structure, and scalability

Agenda

1. Motivation and basics of UVM
2. Core UVM classes
3. Simulation phases and communication
4. Minimal UVM example walkthrough
5. Wrap-up and self-learning

UVM Class Tree



① Sequence Branch: Generating Stimulus

- Goal: Generate the stimulus that drives simulation tests
- Class Hierarchy
 - uvm_transaction: ⇒ Not typically used directly for stimulus generation
 - Base class for all transactions
 - Provides utilities like printing, IDs, and start/stop
 - uvm_sequence_item: ⇒ Commonly called “transactions” 🤪
 - Sequence-specific metadata (e.g., sequence_id, transaction_id)
 - Single unit of stimulus sent from the sequencer to the driver
 - uvm_sequence:
 - A container that defines the order and content of stimulus
 - Runs on a sequencer and sends uvm_sequence_item objects to the driver

Example: Sequence Item to Sequence

Simple uvm_sequence

```
class bus_transaction extends uvm_sequence_item;
  `uvm_object_utils(bus_transaction)

  rand bit [7:0] addr;
  rand bit [31:0] data;
  rand bit rw; // 0 = read, 1 = write

  function new(string name = "bus_transaction");
    super.new(name);
  endfunction

  function string convert2string();
    return $sformatf("addr=0x%0h data=0x%0h rw=%0d", addr, data, rw);
  endfunction
endclass
```

```
class bus_sequence extends uvm_sequence #(bus_transaction);
  `uvm_object_utils(bus_sequence)

  function new(string name = "bus_sequence");
    super.new(name);
  endfunction

  virtual task body();
    bus_transaction tx;

    // Generate 5 transactions
    repeat (5) begin
      tx = bus_transaction::type_id::create("tx");
      start_item(tx);
      assert(tx.randomize());
      finish_item(tx);
    end
  endtask
endclass
```

Handshake items to the driver via the sequencer

This is like handing over one dish (sequence item) from the chef (sequence) to the waiter (sequencer) to deliver to the customer (driver)

② Component Branch: Building The Infrastructure

■ Goal:

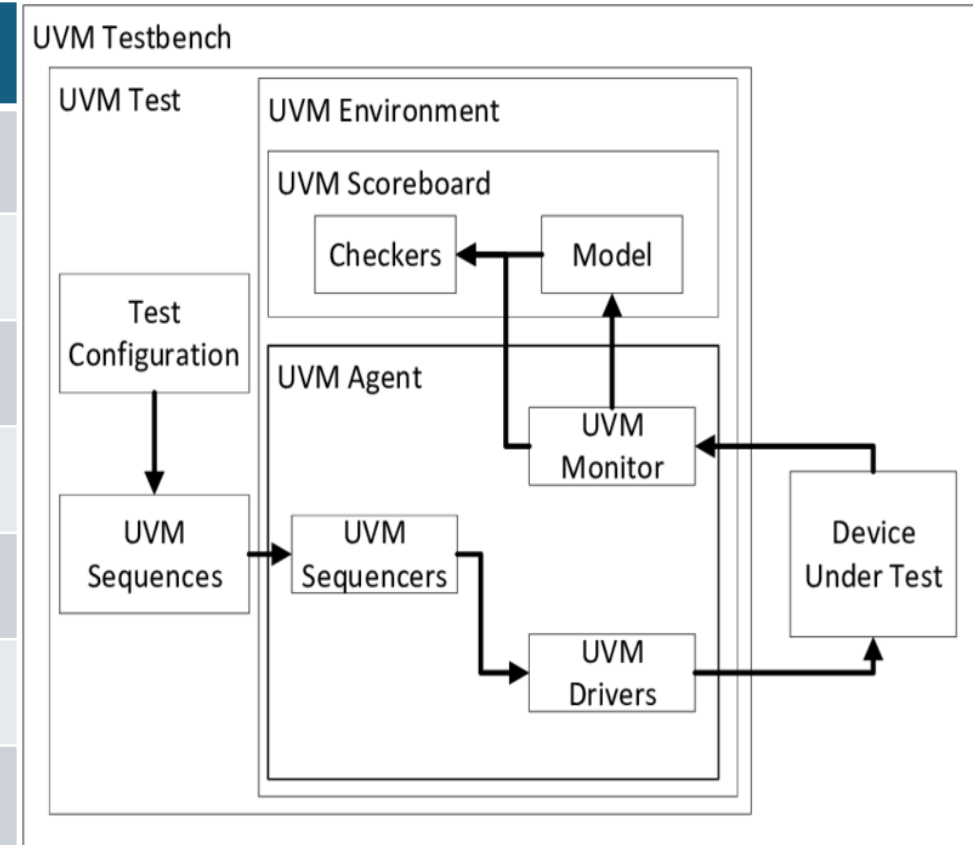
- A uvm_component contains everything that participates in simulation phases
- All building blocks in a UVM testbench inherit from **uvm_component**

■ Simulation Phases

Phase Group	Key Phases	Purpose
Build	build_phase	Create sub-components, read config
Connect	connect_phase	Hook up TLM ports/exports
End of Elaboration	end_of_elaboration_phase	Final structure checks
Simulation Runtime	run_phase	Main stimulus/monitoring loops
Check and Report	check_phase, report_phase	Final checks & reporting

UVM Component Hierarchy

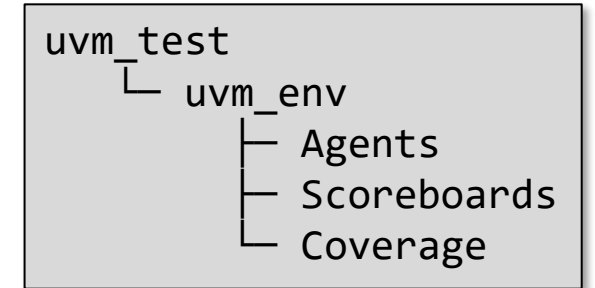
Component	Role
<code>uvm_test</code>	Top-level configuration and sequence selection
<code>uvm_env</code>	Groups agents, scoreboards, coverage
<code>uvm_agent</code>	Encapsulates driver + sequencer + monitor
<code>uvm_driver</code>	Drives DUT pins with transactions
<code>uvm_sequencer</code>	Controls stimulus flow to the driver
<code>uvm_monitor</code>	Observes DUT signals → converts to transactions
<code>uvm_scoreboard</code>	Compares expected vs. actual behavior



Building the Structure (uvm_test and uvm_env)

■ uvm_test

- Topmost component created by UVM
- Configures and activates the environment
- Selects sequences and runtime scenario
- One testbench → many tests via factory and config DB



■ uvm_env

- Groups verification components into a reusable block
- Typical contents:
 - Agents
 - Scoreboard
 - Coverage collectors
 - Register model

Example: Two Tests, One Environment

```
class sanity_test extends uvm_test;
  `uvm_component_utils(sanity_test)

  my_env env;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = my_env::type_id::create("env", this);
  endfunction

  task run_phase(uvm_phase phase);
    phase.raise_objection(this);
    simple_seq seq = simple_seq::type_id::create("seq");
    seq.start(env.agent.seqr);
    phase.drop_objection(this);
  endtask
endclass
```

```
class stress_test extends uvm_test;
  `uvm_component_utils(stress_test)

  my_env env;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    env = my_env::type_id::create("env", this);
  endfunction

  task run_phase(uvm_phase phase);
    phase.raise_objection(this);
    random_seq seq = random_seq::type_id::create("seq");
    seq.num_items = 1000; // more traffic
    seq.start(env.agent.seqr);
    phase.drop_objection(this);
  endtask
endclass
```

```
class my_env extends uvm_env;
  `uvm_component_utils(my_env)

  my_agent agent;
  my_scoreboard sb;

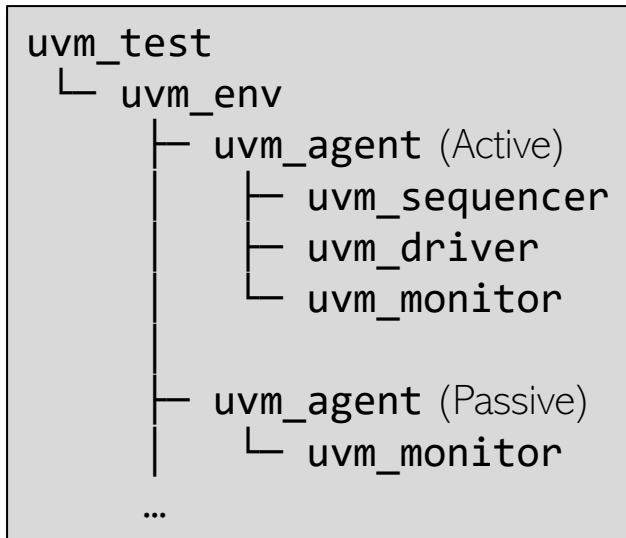
  function new(string name = "my_env", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    agent = my_agent::type_id::create("agent", this);
    sb = my_scoreboard::type_id::create("sb", this);
  endfunction
endclass
```

uvm_agent: Active vs Passive

■ Key Idea:

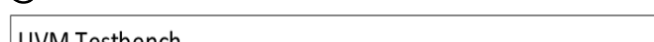
- Reusable component that bundles together all elements needed to **interact with DUT**
- It can operate in active mode (driving stimulus) or passive mode (only observing)
- Only two methods: **new** and **get_is_active**



Feature	Active Agent	Passive Agent
Contains Sequences	Yes	No
Generates Stimulus	Yes: Driver + Sequences	No: Only monitors DUT signals
Used For	Exercising interfaces	Observing existing traffic
Typical Location	Main verification environment	Scoreboarding/Coverage configs
Example Use Case	Stimulating a bus	Checking bus traffic

Stimulus Control: uvm_sequencer

- Key Idea:

- Manages delivery of transactions (sequence_item) from sequences to the driver
 - Handles arbitration when multiple sequences are running
 - Sequence → Sequencer → Driver → DUT
 - Do not confuse with uvm_sequence
- 
- ```

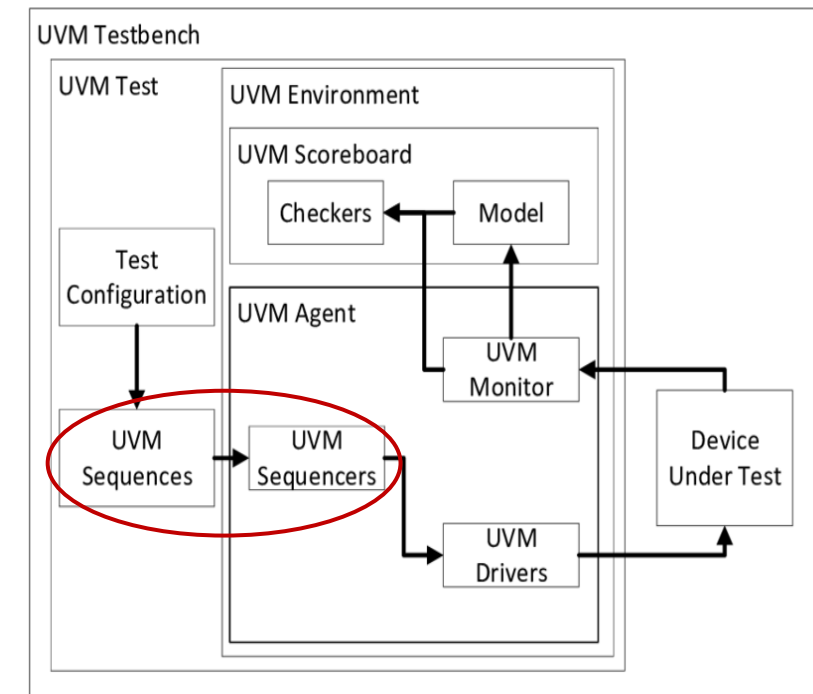
graph TD
 subgraph UVM_Testbench [UVM Testbench]
 subgraph UVM_Test [UVM Test]
 subgraph UVM_Environment [UVM Environment]
 UVM_Sequencer[UVM Sequencer]
 end
 end
 end

```

uvm\_sequence is the the chef, uvm\_sequencer is the waiter

- Responsibilities

- Grants and schedules sequence requests
- Coordinates item handshakes with the driver



# Stimulate (uvm\_driver) and Observe (uvm\_monitor)

## Converts Transactions into Pin-Level Activity

- Purpose
  - Takes transaction objects from the **sequencer**
  - Drives physical signals into the **DUT interface**
- Key Responsibilities
  - Implements the **request-response** handshake
  - Translates **abstract data** → **signal toggles**
- Notes
  - Only included in **active agents**
  - Works tightly with **virtual interfaces**

## Observes DUT Behavior, No Stimulus

- Purpose
  - Passively collects DUT signal activity
  - Reconstructs transactions from signal samples
- Key Responsibilities
  - **Non-intrusive**: no modifying DUT signals
  - Publishes observed transactions via **analysis ports** → Scoreboard, Coverage, Checkers
- Notes
  - Present in **both** active & passive agents
  - Helps achieve **self-checking** and **reusable** environments

Sequence → Sequencer → Driver → DUT → Monitor → Scoreboard/Coverage

# Copilot Analogy: A Restaurant

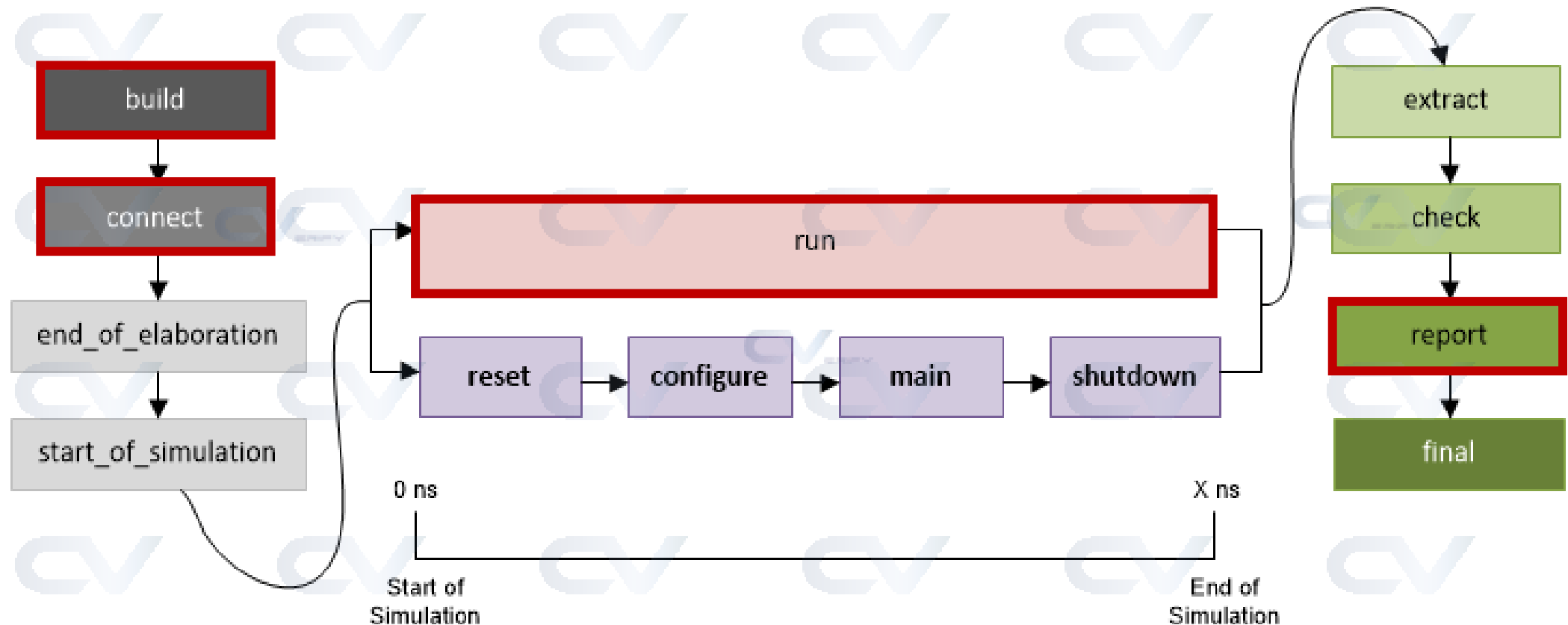
| UVM Concept                    | Restaurant Role                                                                                                                                                            | Description                                 |
|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| <code>uvm_sequence</code>      |  Chef                                                                                     | Prepares the stimulus (transactions)        |
| <code>uvm_sequence_item</code> |  Dish                                                                                     | The actual transaction sent to the driver   |
| <code>uvm_sequencer</code>     |   Waiter | Coordinates delivery of items to the driver |
| <code>uvm_driver</code>        |  Customer                                                                                | Executes the transaction                    |
| <code>uvm_test</code>          |  Menu selector                                                                          | Chooses which sequence (chef) to run        |

**10:00**

# Agenda

1. Motivation and basics of UVM
2. Core UVM classes
3. Simulation phases and communication
4. Minimal UVM example walkthrough
5. Wrap-up and self-learning

# UVM Simulation Phases (uvm\_phase)



Source: <https://www.chipverify.com/uvm/uvm-phases>

# Objections in UVM

The analogy I had in mind for **objections** is a **classroom scenario**:

- Imagine a lecture where the professor is about to end the class.
- Before dismissing, they check: *"Is anyone still working on the exercise?"*
- As long as at least one student has their **hand raised**, the class cannot end.
- Only when **all hands are lowered** does the professor officially dismiss the class.

- **Purpose:** Control when the simulation ends during the run\_phase
  - Default behavior: Simulation would end immediately if no objections are raised
- **Mechanism:**
  - `phase.raise_objection(this);` → signals that a component is still active
  - `phase.drop_objection(this);` → signals that the component has finished
- **End condition:** Simulation continues until all objections are dropped
- **Typical usage:**
  - Sequences raise an objection at start, drop when finished
  - Tests may raise objections to keep simulation alive until checks complete

# Typical Connections in a UVM Testbench

## 1) Stimulus Path (active agent)

Sequence → Sequencer → Driver

## 2) Observation Path

Driver/Monitor → Analysis Ports → Scoreboard

## 3) Configuration Path

Test → Components (via config\_db)

## 4) Environment Assembly

Environment → Agents → Sub-components

# Recap

## ■ Key Points (big-picture summary):

- Phases = WHEN
  - Lifecycle of the testbench (build, connect, run, report).
- Config = WHAT
  - Pass values/handles (e.g., virtual interface) via `uvm_config_db`.
- Connections = HOW
  - Link components (e.g., sequencer ↔ driver) so transactions can flow.

## ■ Closing Line:

- 🙌 With these three ideas, you can already build a minimal but functional UVM testbench

# Agenda

1. Motivation and basics of UVM
2. Core UVM classes
3. Simulation phases and communication
4. Minimal UVM example walkthrough
5. Wrap-up and self-learning

# DUT: Simple FIFO Interface

```
module simple_fifo #(
 parameter WIDTH = 8,
 parameter DEPTH = 4
)(
 input logic clk,
 input logic rst_n,
 input logic wr_en,
 input logic [WIDTH-1:0] wr_data,
 input logic rd_en,
 output logic [WIDTH-1:0] rd_data,
 output logic full,
 output logic empty
);
```

## Spec summary

- Stores data when write enabled
- Outputs oldest stored data when read enabled
- Cannot read and write at the same time
- Generates full and empty status flags

# Transaction Definition

```
class fifo_item extends uvm_sequence_item;
 rand bit wr_en;
 rand bit rd_en;
 rand bit [7:0] data;

 // Status (monitored only)
 bit full;
 bit empty;

 constraint valid_ops {
 !(wr_en && rd_en); // cannot read and write simultaneously
 }

 `uvm_object_utils(fifo_item)

 function new(string name="fifo_item");
 super.new(name);
 endfunction

 function string convert2string();
 return $sformatf("wr=%0b rd=%0b data=0x%0h", wr_en, rd_en, data);
 endfunction
endclass
```

# Sequence and Sequencer

```
class fifo_sequence extends uvm_sequence #(fifo_item);
 `uvm_object_utils(fifo_sequence)

 task body();
 repeat (10) begin
 fifo_item req = fifo_item::type_id::create("req");
 start_item(req);
 assert(req.randomize());
 finish_item(req);
 end
 endtask
endclass
```

Handshake with driver

```
class fifo_sequencer extends uvm_sequencer #(fifo_item);
 `uvm_component_utils(fifo_sequencer)
endclass
```

Why so simple?

Because:

uvm\_sequencer already implements

- Sequence arbitration
- Handshake with driver
- Priority and locking
- Sequencer-Driver TLM ports
- Multiple sequence management

So, for most real environments, we simply **parameterize** it with the transaction type

# Driver Implementation

Virtual interface

```
class fifo_driver extends uvm_driver #(fifo_item);
 `uvm_component_utils(fifo_driver)

 virtual fifo if vif; // Virtual interface to DUT

 function new(string name="fifo_driver", uvm_component parent=null);
 super.new(name, parent);
 endfunction

 task run_phase(uvm_phase phase);
 fifo_item req;
 forever begin
 seq_item_port.get_next_item(req);

 // Drive write
 vif.wr_en <= req.wr_en;
 vif.wr_data <= req.data;

 // Drive read
 vif.rd_en <= req.rd_en;

 @(posedge vif.clk);
 seq_item_port.item_done();
 end
 endtask
endclass
```

Endless  
loop

## Matches handshake with sequence

- Waits until a sequence running on the sequencer has called **start\_item(req)**
- The sequencer ensures that only one item is transferred at a time, avoiding race conditions
- Signals back to the sequencer that the transaction is complete
- This allows the sequence to continue at **finish\_item(req)**

Virtual interface

# Monitor Implementation

```
class fifo_monitor extends uvm_monitor;
 `uvm_component_utils(fifo_monitor)

 virtual fifo if vif;
 uvm_analysis_port #(fifo_item) analysis_port;

 function new(string name="fifo_monitor", uvm_component parent=null);
 super.new(name, parent);
 analysis_port = new("fifo_port ", this);
 endfunction

 task run_phase(uvm_phase phase);
 fifo_item trans;
 forever begin
 @(posedge vif.clk);
 if (vif.wr_en || vif.rd_en) begin
 trans = fifo_item::type_id::create("trans");
 trans.wr_en = vif.wr_en;
 trans.rd_en = vif.rd_en;
 trans.full = vif.full;
 trans.empty = vif.empty;
 trans.data = vif.rd_en ? vif.rd_data : vif.wr_data;
 analysis_port.write(trans); // Publish
 end
 end
 endtask
endclass
```

Endless  
loop

In UVM, the `uvm_analysis_port` is used by passive components (like **monitors**) to **broadcast observed transactions** to anyone who wants to receive them — without knowing who or how many are listening.

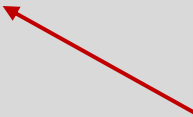
# Scoreboard Draft

```
class my_scoreboard extends uvm_scoreboard;
 `uvm_component_utils(my_scoreboard)

 uvm_analysis_imp #(fifo_tx, my_scoreboard) analysis_export;

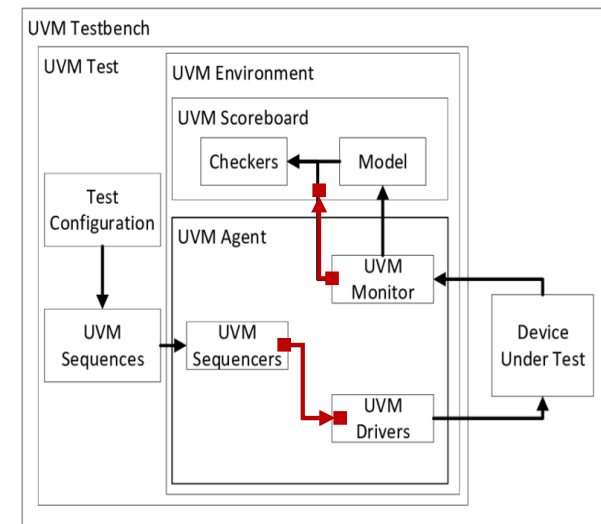
 function new(string name = "my_scoreboard", uvm_component parent);
 super.new(name, parent);
 analysis_export = new("fifo_export", this);
 endfunction

 // Called automatically when monitor writes a transaction
 function void write(fifo_tx t);
 // Example check
 if (/* mismatch */)
 `uvm_error("SCB", "Data mismatch detected")
 endfunction
endclass
```



Analysis port and export  
connected in the environment

# Agent and Environment Connection



```
class fifo_agent extends uvm_agent;
 uvm_sequencer #(fifo_item) sqr;
 fifo_driver drv;
 fifo_monitor mon;

 function new(string name, uvm_component parent);
 super.new(name, parent);
 endfunction

 function void build_phase(uvm_phase phase);
 super.build_phase(phase);
 sqr = uvm_sequencer#(fifo_item)::type_id::create("sqr", this);
 drv = fifo_driver::type_id::create("drv", this);
 mon = fifo_monitor::type_id::create("mon", this);
 endfunction

 function void connect_phase(uvm_phase phase);
 super.connect_phase(phase);
 drv.seq_item_port.connect(sqr.seq_item_export);
 endfunction
endclass
```

Connect sequencer to driver

```
class fifo_env extends uvm_env;
 fifo_agent agent;
 fifo_scoreboard sb;

 function void build_phase(uvm_phase phase);
 super.build_phase(phase);
 agent = fifo_agent::type_id::create("agent", this);
 scb = fifo_scoreboard::type_id::create("scb", this);
 endfunction

 function void connect_phase(uvm_phase phase);
 super.connect_phase(phase);
 agent.mon.analysis_port.connect(scb.analysis_export);
 endfunction
endclass
```

Connect monitor to scoreboard

# A Test

```
class fifo_test extends uvm_test;
 fifo_env env;

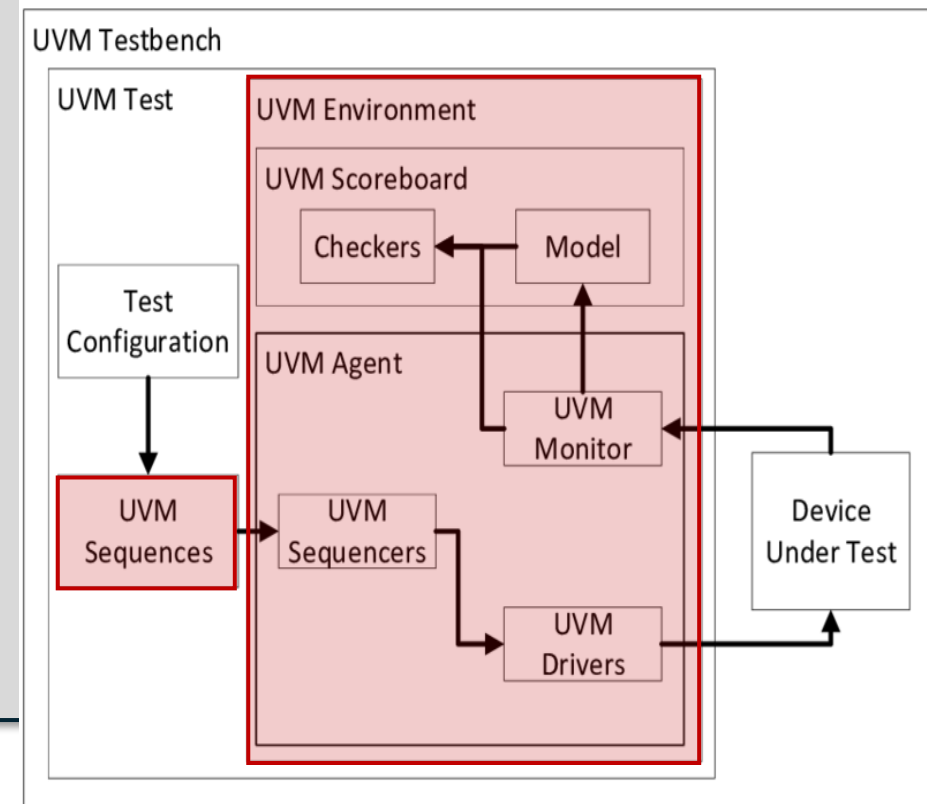
 function new(string name = "fifo_test", uvm_component parent = null);
 super.new(name, parent);
 endfunction

 function void build_phase(uvm_phase phase);
 super.build_phase(phase);
 env = fifo_env::type id::create("env", this);
 endfunction

 task run_phase(uvm_phase phase);
 super.run_phase(phase);

 fifo_basic_seq seq;
 seq = fifo_basic_seq::type id::create("seq");

 phase.raise_objection(this);
 seq.start(env.agent.sqr); // Run sequence in sequencer
 phase.drop_objection(this);
 endtask
endclass
```



# And Finally ... the Testbench Top

```
module tb;
 // Clock + Reset
 logic clk = 0;
 logic rst = 1;

 always #5 clk = ~clk;
 initial begin
 #20 rst = 0;
 end

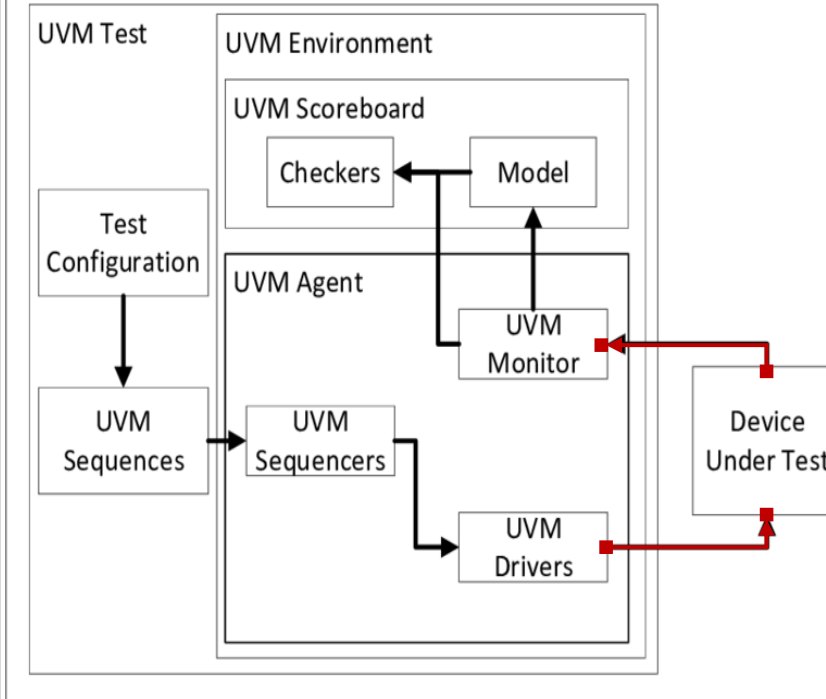
 // Physical interface
 fifo_if vif (clk, rst);

 // DUT Instance
 fifo dut (
 .clk(clk),
 .rst(rst),
 .wr_en(vif.wr_en),
 .rd_en(vif.rd_en),
 .data_in(vif.data_in),
 .data_out(vif.data_out),
 .full(vif.full),
 .empty(vif.empty)
);

 initial begin
 // Publish virtual interface to UVM hierarchy
 uvm_config_db#(virtual fifo_if)::set(null, "uvm_test_top.env.agent.drv", "vif", vif);
 uvm_config_db#(virtual fifo_if)::set(null, "uvm_test_top.env.agent.mon", "vif", vif);

 run_test("fifo_test"); // Kickoff UVM
 end
endmodule
```

UVM Testbench



# Agenda

1. Motivation and basics of UVM
2. Core UVM classes
3. Simulation phases and communication
4. Minimal UVM example walkthrough
5. Wrap-up and self-learning

# Summary of Key Concepts from this Session

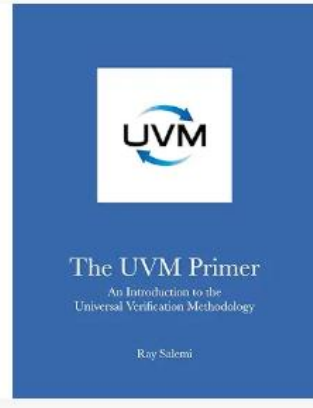
- What is UVM and why is today's de-facto verification methodology
- Class tree: sequence branch + component branch
- UVM objects and components, and how to connect them
- A simple example: FIFO

# Link to Next Week

- **Week 09: Advanced and practical UVM**
  - Move from “toy” UVM examples to scalable, configurable environments
  - Learn UVM service mechanisms (factory, config, report)
  - Understand communication via TLM (analysis ports/FIFOs)
  - Multiple agents and coordinate via virtual sequencers

# Self-Learning Corner

## ■ Books



## ■ Webs

- <https://verificationguide.com/uvm/uvm-tutorial>
- <https://www.chipverify.com/tutorials/uvm>
- <https://www.doulos.com/knowhow/systemverilog/uvm/easier-uvm>

“Verification is not about proving you’re right. It's about proving you’re not wrong.”

Industry saying